

Competitive Programming Laboratory

BTCOC606 | TY-A & B | Semester VI | 2025-26

KBP College of Engineering, Satara

Bug-Fix Summary

Prog #	Program Title	Status
1	Read N Integers and Print Their Squares	Prompts Added
2	Weird or Not Weird	Prompts Added
3	Formatted String and Integer Output	Correct
4	Print First 10 Multiples of N	Prompts Added
5	Read Lines Until EOF and Print Numbered	Correct
6	Print Substring from start to end-1	Prompts Added
7	Check if a String is Palindrome	Prompts Added
8	Tokenize a String	Prompts Added
9	Check if a Number is Prime	Prompts Added
10	Array Storage and Linear Search	Prompts Added
11	Java List - Insert and Delete Operations	Fixed + Prompts
12	Phone Book using HashMap	Fixed + Prompts
13	Generic Method to Print Any Array	Correct
14	Sort Students by CGPA, Name, and ID	Prompts Added
15	Inheritance - Single and Multilevel	Correct
16	Abstract Class	Correct
17	Interfaces	Correct
18	Method Overloading	Correct
19	instanceof Operator	Correct
20	Exception Handling	Correct
21	Singleton Design Pattern	Correct
22	The 3n+1 Problem	Prompts Added
23	Jolly Jumpers	Correct
24	WERTYU Keyboard Decoder	Correct
25	Primary Arithmetic - Count Carry Operations	Fixed + Prompts
26	Reverse and Add (Find Palindrome)	Prompts Added

Program 1: Read N Integers and Print Their Squares [PROMPTS ADDED]

Problem Statement:

Read N integers from stdin and print each on a new line. Then find and print the square of each of the N numbers.

Fix / Change Notes:

Added missing input prompts: 'Enter value of N:' and 'Enter number i:' inside the loop.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog1 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter value of N: ");

        int n = sc.nextInt();

        int[] arr = new int[n];

        // Step 1: Read n integers
        for (int i = 0; i < n; i++) {

            System.out.print("Enter number " + (i + 1) + ": ");

            arr[i] = sc.nextInt();

        }

        // Step 2: Print each number
        System.out.println("Numbers:");

        for (int i = 0; i < n; i++) {

            System.out.println(arr[i]);

        }

        // Step 3: Print squares
        System.out.println("Squares:");

        for (int i = 0; i < n; i++) {

            System.out.println(arr[i] * arr[i]);

        }

    }

}
```

Program 2: Weird or Not Weird [PROMPTS ADDED]

Problem Statement:

Given integer N: Odd -> Weird | Even 2-5 -> Not Weird | Even 6-20 -> Weird | Even > 20 -> Not Weird

Fix / Change Notes:

Added missing input prompt: 'Enter a number (N):' before reading N.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog2 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number (N): ");

        int n = sc.nextInt();

        if (n % 2 != 0) {

            System.out.println("Weird");

        } else if (n >= 2 && n <= 5) {

            System.out.println("Not Weird");

        } else if (n >= 6 && n <= 20) {

            System.out.println("Weird");

        } else {

            System.out.println("Not Weird");

        }

    }

}
```

Program 3: Formatted String and Integer Output [OK]

Problem Statement:

Each line has a String (max 10 chars) and an integer (0-999). Print the String left-justified in 15 chars and the integer zero-padded to 3 digits.

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog3 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        while (sc.hasNext()) {

            String s = sc.next();

            int num = sc.nextInt();

            // %-15s: left-justify in 15 chars | %03d: zero-pad to 3 digits

            System.out.printf("%-15s%03d\n", s, num);

        }

    }

}
```

Program 4: Print First 10 Multiples of N [PROMPTS ADDED]

Problem Statement:

Given integer N, print its first 10 multiples in format: N x i = result

Fix / Change Notes:

Added missing input prompt: 'Enter a number (N):' before reading N.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog4 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number (N): ");

        int n = sc.nextInt();

        for (int i = 1; i <= 10; i++) {

            System.out.println(n + " x " + i + " = " + (n * i));

        }

    }

}
```

Program 5: Read Lines Until EOF and Print Numbered [OK]

Problem Statement:

Read lines until EOF, then print all lines with their line numbers.

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
import java.util.Scanner;

import java.util.ArrayList;

public class Prog5 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        ArrayList<String> lines = new ArrayList<>();

        while (sc.hasNextLine()) {

            lines.add(sc.nextLine());

        }

        for (int i = 0; i < lines.size(); i++) {

            System.out.println((i + 1) + " " + lines.get(i));

        }

    }

}
```

Program 6: Print Substring from start to end-1 [PROMPTS ADDED]

Problem Statement:

Given string S and indices start and end, print the substring from index start to end-1.

Fix / Change Notes:

Added missing input prompts: 'Enter the string:', 'Enter start index:', 'Enter end index:'.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog6 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the string: ");

        String s = sc.nextLine();

        System.out.print("Enter start index: ");

        int start = sc.nextInt();

        System.out.print("Enter end index : ");

        int end = sc.nextInt();

        // substring(start, end) returns chars from start up to (end-1)

        System.out.println("Substring: " + s.substring(start, end));

    }

}
```

Program 7: Check if a String is Palindrome [PROMPTS ADDED]

Problem Statement:

Check whether the given string is a palindrome (reads same forwards and backwards).

Fix / Change Notes:

Added missing input prompt: 'Enter a string:' before reading the string.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog7 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");

        String s      = sc.nextLine();

        String reversed = new StringBuilder(s).reverse().toString();

        if (s.equals(reversed)) {

            System.out.println(s + " is a Palindrome");

        } else {

            System.out.println(s + " is NOT a Palindrome");

        }

    }

}
```


Program 8: Tokenize a String [PROMPTS ADDED]

Problem Statement:

Split the string into tokens (consecutive letters only). Print count of tokens, then each token.

Fix / Change Notes:

Added missing input prompt: 'Enter a string to tokenize:' before reading the string.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog8 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string to tokenize: ");

        String s = sc.nextLine();

        // Split on anything that is NOT a letter
        String[] tokens = s.split("[^a-zA-Z]+");

        int count = 0;

        for (String t : tokens) {

            if (!t.isEmpty()) count++;

        }

        System.out.println("Token count: " + count);

        System.out.println("Tokens:");

        for (String t : tokens) {

            if (!t.isEmpty()) System.out.println(t);

        }

    }

}
```

Program 9: Check if a Number is Prime [PROMPTS ADDED]

Problem Statement:

Check whether the given number is prime (greater than 1, no divisors other than 1 and itself).

Fix / Change Notes:

Added missing input prompt: 'Enter a number:' before reading N.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog9 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");

        int n = sc.nextInt();

        if (n < 2) {

            System.out.println(n + " is NOT Prime");

            return;

        }

        boolean isPrime = true;

        for (int i = 2; i <= Math.sqrt(n); i++) {

            if (n % i == 0) { isPrime = false; break; }

        }

        System.out.println(isPrime ? n + " is Prime" : n + " is NOT Prime");

    }

}
```

Program 10: Array Storage and Linear Search [PROMPTS ADDED]

Problem Statement:

Accept n numbers in an array, print each, then search for a key. Print 'Found' or 'Not Found'.

Fix / Change Notes:

Added missing input prompts: 'Enter array size:', 'Enter element i:' in loop, and 'Enter key to search:' before reading key.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog10 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");

        int n      = sc.nextInt();

        int[] arr  = new int[n];

        for (int i = 0; i < n; i++) {

            System.out.print("Enter element " + (i + 1) + ": ");

            arr[i] = sc.nextInt();

        }

        System.out.println("Array elements:");

        for (int x : arr) System.out.println(x);

        System.out.print("Enter key to search: ");

        int key      = sc.nextInt();

        boolean found = false;

        for (int x : arr) {

            if (x == key) { found = true; break; }

        }

        System.out.println(found ? "Found" : "Not Found");

    }

}
```

Program 11: Java List - Insert and Delete Operations [FIXED + PROMPTS]

Problem Statement:

Given a list of N integers, perform Q queries: 'Insert x y' inserts value y at index x, 'Delete x' removes element at index x. Print the final list.

Fix / Change Notes:

BUG FIXED (previous): Read entire query line and split into tokens. PROMPTS ADDED: 'Enter list size:', 'Enter element i:', 'Enter number of queries:', 'Enter query i:' for each operation.

Corrected Java Code:

```
import java.util.*;

public class Prog11 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter list size: ");

        int n = sc.nextInt();

        List<Integer> list = new ArrayList<>();

        for (int i = 0; i < n; i++) {

            System.out.print("Enter element " + (i + 1) + ": ");

            list.add(sc.nextInt());

        }

        System.out.print("Enter number of queries: ");

        int q = sc.nextInt();

        sc.nextLine(); // consume newline after q

        for (int i = 0; i < q; i++) {

            // Each query is a SINGLE line: "Insert x y" or "Delete x"

            System.out.print("Enter query " + (i + 1) + " (Insert x y / Delete x): ");

            String[] parts = sc.nextLine().trim().split("\\s+");

            String op = parts[0];

            if (op.equals("Insert")) {

                int x = Integer.parseInt(parts[1]); // index

                int y = Integer.parseInt(parts[2]); // value

                list.add(x, y);

            } else if (op.equals("Delete")) {

                int x = Integer.parseInt(parts[1]); // index

                list.remove(x);

            }

        }

    }

}
```

```
        // Print final list space-separated
        System.out.print("Final list: ");
        for (int i = 0; i < list.size(); i++) {
            System.out.print(list.get(i));
            if (i < list.size() - 1) System.out.print(" ");
        }
        System.out.println();
    }
}
```

Program 12: Phone Book using HashMap [FIXED + PROMPTS]

Problem Statement:

Store n name-phone entries. For each query name, print 'name=number' if found, else 'Not found'.

Fix / Change Notes:

BUG FIXED (previous): Each entry is one line 'name phone'. PROMPTS ADDED: 'Enter number of entries:', 'Enter entry i (name phone):', and 'Enter name to search:' for each query.

Corrected Java Code:

```
import java.util.*;

public class Prog12 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of entries: ");

        int n      = sc.nextInt();

        sc.nextLine();    // consume newline after n

        HashMap<String, String> phoneBook = new HashMap<>();

        // Each entry is on ONE line: "name phonenumber"
        for (int i = 0; i < n; i++) {

            System.out.print("Enter entry " + (i + 1) + " (name phone): ");

            String[] parts = sc.nextLine().trim().split("\\s+");

            String name     = parts[0];

            String phone    = parts[1];

            phoneBook.put(name, phone);

        }

        // Process queries until EOF

        System.out.println("Enter names to search (Ctrl+D / Ctrl+Z to stop):");

        while (sc.hasNextLine()) {

            String query = sc.nextLine().trim();

            if (query.isEmpty()) continue;

            if (phoneBook.containsKey(query))

                System.out.println(query + "=" + phoneBook.get(query));

            else

                System.out.println("Not found");

        }

    }

}
```

Program 13: Generic Method to Print Any Array [OK]

Problem Statement:

Write a single generic method printArray that works for both Integer and String arrays.

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
public class Prog13 {  
    // <T> makes this method work for ANY reference data type  
    public static <T> void printArray(T[] arr) {  
        for (T item : arr) {  
            System.out.print(item + " ");  
        }  
        System.out.println();  
    }  
  
    public static void main(String[] args) {  
        Integer[] intArr = {1, 2, 3, 4, 5};  
        String[] strArr = {"Hello", "World", "Java"};  
  
        System.out.println("Integer Array:");  
        printArray(intArr);  
  
        System.out.println("String Array:");  
        printArray(strArr);  
    }  
}
```

Program 14: Sort Students by CGPA, Name, and ID [PROMPTS ADDED]

Problem Statement:

Sort students: (1) CGPA descending, (2) same CGPA -> name alphabetically, (3) same name -> by ID. Print first name of each student.

Fix / Change Notes:

Added missing input prompts: 'Enter number of students:' and 'Enter student i (ID Name CGPA):' inside the loop.

Corrected Java Code:

```
import java.util.*;

public class Prog14 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of students: ");

        int n      = sc.nextInt();

        sc.nextLine();

        List<String[]> students = new ArrayList<>();

        for (int i = 0; i < n; i++) {

            System.out.print("Enter student " + (i + 1) + " (ID Name CGPA): ");

            String id  = sc.next();

            String name = sc.next();

            double cgpa = sc.nextDouble();

            students.add(new String[]{id, name, String.valueOf(cgpa)});

        }

        students.sort((a, b) -> {

            double ca = Double.parseDouble(a[2]);

            double cb = Double.parseDouble(b[2]);

            if (cb != ca)          return Double.compare(cb, ca);    // CGPA desc

            if (!a[1].equals(b[1])) return a[1].compareTo(b[1]);    // name asc

            return a[0].compareTo(b[0]);                            // ID asc

        });

        System.out.println("Students sorted by CGPA (descending):");

        for (String[] s : students) System.out.println(s[1]);

    }

}
```


Program 15: Inheritance - Single and Multilevel [OK]

Problem Statement:

Demonstrate Single Inheritance (Dog extends Animal) and Multilevel Inheritance (Puppy extends Dog).

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
class Animal {
    void eat() { System.out.println("Animal eats"); }
}

class Dog extends Animal {
    void bark() { System.out.println("Dog barks"); }
}

class Puppy extends Dog {
    void weep() { System.out.println("Puppy weeps"); }
}

public class Prog15 {
    public static void main(String[] args) {
        System.out.println("-- Single Inheritance --");

        Dog d = new Dog();
        d.eat();    // inherited from Animal
        d.bark();   // Dog's own method

        System.out.println("-- Multilevel Inheritance --");

        Puppy p = new Puppy();
        p.eat();    // inherited from Animal
        p.bark();   // inherited from Dog
        p.weep();   // Puppy's own method
    }
}
```

Program 16: Abstract Class [OK]

Problem Statement:

Create abstract class Shape with abstract method area(). Implement in Circle and Rectangle.

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
abstract class Shape {
    String color;

    abstract double area();    // subclasses MUST implement this

    void display() {
        System.out.println("Color: " + color);
        System.out.println("Area : " + area());
    }
}

class Circle extends Shape {
    double radius;

    Circle(String color, double radius) {
        this.color = color;
        this.radius = radius;
    }

    double area() { return Math.PI * radius * radius; }
}

class Rectangle extends Shape {
    double length, width;

    Rectangle(String color, double l, double w) {
        this.color = color;
        length = l;
        width = w;
    }

    double area() { return length * width; }
}

public class Prog16 {
    public static void main(String[] args) {
        new Circle("Red", 5).display();
        new Rectangle("Blue", 4, 6).display();
    }
}
```

Program 17: Interfaces [OK]

Problem Statement:

Create interfaces Drawable and Colorable. Implement both in a Circle class.

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
interface Drawable {
    void draw();
}

interface Colorable {
    void fillColor(String color);
}

class Circle implements Drawable, Colorable {
    public void draw() {
        System.out.println("Drawing Circle");
    }

    public void fillColor(String color) {
        System.out.println("Circle filled with " + color);
    }
}

public class Prog17 {
    public static void main(String[] args) {
        Circle c = new Circle();
        c.draw();
        c.fillColor("Red");
    }
}
```

Program 18: Method Overloading [OK]

Problem Statement:

Demonstrate method overloading: same method name 'add' with different parameter types and counts.

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
public class Prog18 {  
    int    add(int a, int b)          { return a + b; }  
    double add(double a, double b)   { return a + b; }  
    int    add(int a, int b, int c)   { return a + b + c; }  
  
    public static void main(String[] args) {  
        Prog18 obj = new Prog18();  
        System.out.println("add(2, 3)      = " + obj.add(2, 3));  
        System.out.println("add(2.5, 3.5)  = " + obj.add(2.5, 3.5));  
        System.out.println("add(1, 2, 3)   = " + obj.add(1, 2, 3));  
    }  
}
```

Program 19: instanceof Operator [OK]

Problem Statement:

Demonstrate instanceof to check if an object is an instance of a class or its parent.

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
class Animal { }

class Dog    extends Animal { }

class Cat    extends Animal { }

public class Prog19 {

    public static void main(String[] args) {

        Dog dog = new Dog();

        System.out.println(dog instanceof Dog);    // true

        System.out.println(dog instanceof Animal); // true

        System.out.println(dog instanceof Cat);    // false

        Animal a = new Dog();

        if (a instanceof Dog) {

            System.out.println("a is actually a Dog");

        }

    }

}
```

Program 20: Exception Handling [OK]

Problem Statement:

Demonstrate try-catch-finally. Accept two numbers, divide them, handle ArithmeticException.

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog20 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {

            System.out.print("Enter numerator : ");

            int a = sc.nextInt();

            System.out.print("Enter denominator: ");

            int b = sc.nextInt();

            if (b == 0)

                throw new ArithmeticException("Cannot divide by zero!");

            System.out.println("Result: " + (a / b));

        } catch (ArithmeticException e) {

            System.out.println("Caught: " + e.getMessage());

        } finally {

            System.out.println("finally block always runs.");

        }

    }

}
```

Program 21: Singleton Design Pattern [OK]

Problem Statement:

Implement Singleton: only one object ever created. Use private constructor and static getInstance().

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
public class Singleton {  
    private static Singleton instance = null;  
  
    // Private constructor prevents external instantiation  
    private Singleton() {  
        System.out.println("Singleton instance created.");  
    }  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
  
    public void showMessage() {  
        System.out.println("Hello from Singleton!");  
    }  
  
    public static void main(String[] args) {  
        Singleton obj1 = Singleton.getInstance();  
        obj1.showMessage();  
        Singleton obj2 = Singleton.getInstance();  
        System.out.println("Same instance? " + (obj1 == obj2)); // true  
    }  
}
```

Program 22: The 3n+1 Problem [PROMPTS ADDED]

Problem Statement:

Start with n . If even divide by 2; if odd do $3n+1$. Repeat until $n=1$. Print full sequence.

Fix / Change Notes:

Added missing input prompt: 'Enter a number (N):' before reading N.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog22 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number (N): ");

        int n = sc.nextInt();

        System.out.print("Sequence: " + n);

        while (n != 1) {

            if (n % 2 == 0)

                n = n / 2;

            else

                n = 3 * n + 1;

            System.out.print(" " + n);

        }

        System.out.println();

    }

}
```


Program 23: Jolly Jumpers [OK]

Problem Statement:

A sequence is 'jolly' if absolute differences between successive elements cover all values 1 to n-1.

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog23 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        while (sc.hasNextInt()) {

            int n      = sc.nextInt();

            int[] seq  = new int[n];

            for (int i = 0; i < n; i++) seq[i] = sc.nextInt();

            if (n == 1) { System.out.println("Jolly"); continue; }

            boolean[] seen = new boolean[n];

            boolean jolly = true;

            for (int i = 1; i < n; i++) {

                int diff = Math.abs(seq[i] - seq[i - 1]);

                if (diff < 1 || diff >= n || seen[diff]) {

                    jolly = false; break;

                }

                seen[diff] = true;

            }

            System.out.println(jolly ? "Jolly" : "Not jolly");

        }

    }

}
```

Program 24: WERTYU Keyboard Decoder [OK]

Problem Statement:

Hands placed one key right on keyboard. Decode by replacing each char with the one to its left.

Code Review Notes:

No errors found. Code is correct.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog24 {

    public static void main(String[] args) {

        String keyboard =

            "`1234567890-=QWERTYUIOP[]\ASDFGHJKL;'ZXCVBNM,./";

        Scanner sc = new Scanner(System.in);

        while (sc.hasNextLine()) {

            String line = sc.nextLine();

            StringBuilder result = new StringBuilder();

            for (char c : line.toCharArray()) {

                int idx = keyboard.indexOf(c);

                if (idx > 0)

                    result.append(keyboard.charAt(idx - 1));

                else

                    result.append(c);

            }

            System.out.println(result);

        }

    }

}
```

Program 25: Primary Arithmetic - Count Carry Operations [FIXED + PROMPTS]

Problem Statement:

Count carry operations when adding two numbers digit by digit (right to left). Stop at '0 0'.

Fix / Change Notes:

BUG FIXED (previous): Added trailing carry check after loop. PROMPTS ADDED: 'Enter two numbers (0 0 to stop):' before each pair.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog25 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        while (true) {

            System.out.print("Enter two numbers (0 0 to stop): ");

            if (!sc.hasNextLong()) break;

            long a = sc.nextLong(), b = sc.nextLong();

            if (a == 0 && b == 0) break;

            int carry = 0, count = 0;

            while (a > 0 || b > 0) {

                int sum = (int)(a % 10) + (int)(b % 10) + carry;

                carry = sum / 10;

                if (carry > 0) count++;

                a /= 10;

                b /= 10;

            }

            // FIX: count trailing carry after both numbers are exhausted

            if (carry > 0) count++;

            if (count == 0) System.out.println("No carry operation.");

            else if (count == 1) System.out.println("1 carry operation.");

            else

                System.out.println(count + " carry operations.");

        }

    }

}
```

Program 26: Reverse and Add (Find Palindrome) [PROMPTS ADDED]

Problem Statement:

Reverse digits of a number and add to original. Repeat until palindrome. Print steps and result.

Fix / Change Notes:

Added missing input prompts: 'Enter number of test cases:' and 'Enter number:' for each test case.

Corrected Java Code:

```
import java.util.Scanner;

public class Prog26 {

    static long reverse(long n) {
        long rev = 0;
        while (n > 0) { rev = rev * 10 + n % 10; n /= 10; }
        return rev;
    }

    static boolean isPalindrome(long n) {
        return n == reverse(n);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of test cases: ");
        int t = sc.nextInt();
        while (t-- > 0) {
            System.out.print("Enter number: ");
            long p = sc.nextLong();
            int steps = 0;
            while (!isPalindrome(p)) {
                p = p + reverse(p);
                steps++;
            }
            System.out.println(steps + " " + p);
        }
    }
}
```